

Módulo 5: Modelos Grandes de Lenguaje

Tema 2: Entrenamiento de LLMs

Diplomado en Inteligencia Artificial

Facultad de Ingeniería, UNAM

2026-2

Objetivos del Tema 2

Al finalizar este tema, el participante será capaz de:

- Comprender la arquitectura interna de un LLM moderno.
- Explicar el papel del mecanismo de atención en los Transformers.
- Describir el proceso completo de entrenamiento de un LLM.
- Analizar los retos computacionales y de datos del entrenamiento a gran escala.

Entrenar un LLM no es trivial

Entrenar un LLM implica:

- Miles de millones de parámetros
- Volúmenes masivos de datos
- Costos computacionales elevados

Por ello, el entrenamiento requiere decisiones arquitectónicas y estratégicas.

Separación conceptual importante

Es fundamental distinguir entre:

- **Arquitectura del modelo**
- **Proceso de entrenamiento**
- **Uso del modelo entrenado**

Este tema se enfoca en los dos primeros.

Arquitectura Transformer

La mayoría de los LLMs modernos se basan en:

- La arquitectura Transformer
- Propuesta para manejar dependencias largas en secuencias

Componentes principales del Transformer

Un Transformer está compuesto por:

- Embeddings
- Capas de atención
- Redes feedforward
- Normalización y conexiones residuales

Representación del texto

Antes de procesar texto:

- El texto se tokeniza
- Cada token se representa como un vector

Estos vectores son la entrada al modelo.

Mecanismo de atención

La atención permite al modelo:

- Ponderar la relevancia de otros tokens
- Capturar contexto global

Atención auto-regresiva

En LLMs generativos:

- Cada token atiende solo a tokens anteriores
- Se preserva la causalidad

Multi-head attention

La atención múltiple permite:

- Aprender distintos tipos de relaciones
- Procesar información en paralelo

¿Autoencoders en LLMs?

Aunque los LLMs generativos son auto-regresivos:

- Comparten ideas con autoencoders
- Especialmente en la representación latente

Objetivo del entrenamiento

El entrenamiento busca:

- Minimizar el error de predicción del siguiente token
- Ajustar millones o billones de parámetros

Aprendizaje auto-supervisado

Los LLMs se entrenan mediante:

- Texto sin etiquetas explícitas
- El propio texto genera las señales de aprendizaje

Datos de entrenamiento

Los datos suelen incluir:

- Libros
- Artículos
- Código
- Contenido web

Preprocesamiento de datos

Antes del entrenamiento:

- Limpieza
- Filtrado
- Tokenización
- Balance de dominios

Función de pérdida

La función de pérdida típica es:

- Cross-entropy

Mide qué tan bien el modelo predice el siguiente token.

Optimización

El entrenamiento utiliza:

- Descenso por gradiente
- Optimizadores como Adam

Ajuste de hiperparámetros

Decisiones críticas:

- Tasa de aprendizaje
- Tamaño de batch
- Número de capas
- Dimensión de embeddings

Escalamiento del entrenamiento

Entrenar LLMs requiere:

- Paralelismo de datos
- Paralelismo de modelo
- Infraestructura distribuida

Costos computacionales

Aspectos a considerar:

- Consumo energético
- Tiempo de entrenamiento
- Acceso a hardware especializado

Entrenamiento eficiente

Estrategias comunes:

- Mixed precision
- Gradient checkpointing
- Curriculum learning

Riesgos del entrenamiento

Durante el entrenamiento, pueden aparecer:

- Sobreajuste
- Sesgos en los datos
- Comportamientos no deseados

Evaluación durante el entrenamiento

Se evalúa:

- Pérdida
- Perplejidad
- Comportamiento cualitativo

Práctica 1: Modelo de lenguaje (baseline)

Meta: entrenar un modelo que aprenda a predecir el siguiente token.

Práctica 1 LLMs

En esta práctica:

- Construiremos un modelo carácter-a-carácter.
- Entrenaremos con *next-token prediction*.
- Generaremos texto y analizaremos errores.

Definición del problema

Dado un contexto de caracteres:

$$x_{t-k}, \dots, x_{t-1}$$

predecir el siguiente carácter:

$$x_t$$

Esto replica el objetivo base de un LLM, en versión reducida.

Datos y tokenización

Dataset: un archivo de texto corto (ej., artículo, cuento, README).

Tokenización:

- Vocabulario = conjunto de caracteres observados
- Mapeo: caracter $\leftrightarrow$ id

Ventanas de entrenamiento

Se generan pares (entrada, objetivo):

- Entrada: una ventana de longitud L
- Objetivo: el mismo segmento corrido una posición

Ejemplo:

Entrada: "hola "

Objetivo: "ola m"

Arquitectura baseline

Modelo simple (sin atención):

- Embedding
- RNN/GRU (o MLP simple)
- Capa lineal a vocabulario

Interpretación: aprende dependencias locales y patrones frecuentes.

Pérdida y entrenamiento

Función de pérdida: cross-entropy por paso temporal.

Entrenamiento:

- Mini-batches de ventanas
- Optimización con Adam
- Monitoreo de pérdida (loss)

Generación de texto

Decodificación:

- Greedy: elegir el token más probable
- Sampling: muestrear con temperatura

Pregunta clave: ¿cómo cambian los resultados al variar la temperatura?

Preguntas de análisis

- ¿Qué patrones aprende el modelo?
- ¿Qué errores comete y por qué?
- ¿Qué cambia si aumentamos L (contexto)?
- ¿Qué cambia con más datos o más épocas?

Entregables

- Gráfica o tabla de pérdida por época (o por iteraciones)
- 3 muestras de texto generado (temperatura baja/media/alta)
- Comentario técnico: limitaciones observadas

Práctica 2: Contexto largo y degradación

Meta: observar qué ocurre cuando aumentamos el tamaño del contexto.
Práctica 2 LLMs

En esta práctica:

- Entrenaremos el mismo modelo con distintos contextos L .
- Mediremos pérdida/perplejidad y calidad de generación.
- Analizaremos limitaciones de modelos secuenciales.

Hipótesis

- A mayor L , el modelo tiene más información disponible.
- Pero también aumenta la dificultad de entrenamiento y la estabilidad.

Pregunta: ¿mejora siempre el desempeño cuando crece el contexto?

Diseño experimental

Entrenar el mismo modelo (baseline) variando L :

- $L \in \{32, 64, 128, 256\}$
- Misma cantidad de épocas
- Mismo optimizador y tasa de aprendizaje

Medir:

- Pérdida y perplejidad en validación
- Calidad cualitativa del texto generado

Qué observar

- ¿Se estabiliza el entrenamiento?
- ¿Cambia la perplejidad?
- ¿Mejora la coherencia global del texto?
- ¿Aparecen repeticiones o colapsos?

Discusión: limitaciones

Si el modelo no aprovecha contexto largo, puede indicar:

- Dificultad para propagar información (señal/gradiente)
- Capacidad insuficiente (tamaño del estado oculto)
- Optimización más difícil

Esto motiva arquitecturas con acceso directo al contexto: **atención**.

Entregables

- Tabla/figura: L vs pérdida (y/o perplejidad) de validación
- 2 muestras generadas por cada L (misma temperatura)
- Conclusión técnica: ¿qué cambia y por qué?

Práctica 3: Mini-Transformer (LLM reducido)

Meta: entrenar un Transformer causal para predecir el siguiente token.
Práctica 3 LLMs

En esta práctica:

- Implementaremos self-attention con máscara causal.
- Entrenaremos con cross-entropy (next-token).
- Generaremos texto con sampling y temperatura.

Recordatorio: objetivo de entrenamiento

Dado un contexto:

$$x_1, x_2, \dots, x_{t-1}$$

predecir:

$$x_t$$

Causalidad: el token t no puede usar información del futuro.

Bloques del Transformer causal

Componentes por bloque:

- Multi-Head Self-Attention (con máscara causal)
- Feedforward (MLP por posición)
- Residual + LayerNorm

Atención causal

La máscara triangular asegura:

token t atiende solo a $\{1, \dots, t\}$

Esto habilita la generación auto-regresiva.

Positional encoding

Como la atención no impone orden:

- Se agrega información de posición
- Sin posición, el modelo no distingue permutaciones

Diseño de experimento

Entrenar un mini-Transformer con:

- 2–4 capas
- 2–4 cabezas
- contexto L (ej., 128)

Medir:

- pérdida / perplejidad
- coherencia del texto generado

Entregables

- Tabla: configuración (capas/cabezas/dim) vs val_loss
- 3 muestras de texto (temperatura baja/media/alta)
- Comentario: ¿qué mejora hay respecto al baseline GRU?

Discusión

Comparar con GRU del Ejercicio 1–2:

- 1 ¿A igual L , el Transformer aprovecha mejor el contexto?
- 2 ¿Se ve mayor coherencia local/global?
- 3 ¿Qué pasa cuando aumentan L a 256?

Práctica 4: Hiperparámetros en entrenamiento de LLMs

Meta: comprender que entrenar LLMs es un problema de ingeniería.

Práctica 4 LLMs

Exploraremos cómo cambian:

- convergencia (pérdida/perplejidad),
- estabilidad (divergencia/gradientes),
- costo (tiempo/memoria),
- calidad cualitativa del texto.

Hiperparámetros bajo estudio

Se evaluarán (una cuadrícula pequeña):

- Tasa de aprendizaje (LR)
- Batch size
- Profundidad (número de capas)
- Tamaño del modelo (dimensión de embedding)

Diseño experimental

- Mantener datos fijos
- Mantener tokenización fija
- Cambiar un hiperparámetro por vez (o cuadrícula controlada)

Principio: para atribuir efectos, no mezclar cambios.

Métricas a registrar

- Train loss y Val loss
- Perplejidad (aprox.): $\exp(\text{Val loss})$
- Tiempo por época
- Señales de inestabilidad (NaN, explosión de gradiente)

Preguntas guía

- ¿Qué hiperparámetros afectan más la estabilidad?
- ¿Qué combinación da el mejor trade-off costo/desempeño?
- ¿Qué cambios mejoran la generación (cualitativo)?

Entregables

- Tabla comparativa de configuraciones y métricas
- Evidencia de al menos 1 corrida inestable (y explicación)
- Recomendación: configuración óptima para CPU vs GPU

Discusión

- 1 Explicar en términos de ingeniería:
- 2 ¿Por qué un LR alto puede causar divergencia?
- 3 ¿Por qué el batch cambia la estabilidad y velocidad (ruido del gradiente)?
- 4 ¿Por qué un modelo más grande puede mejorar, pero cuesta más y puede sobreajustar si el texto es pequeño?
- 5 ¿Qué combinación escogerían para:
 - CPU en laboratorio?
 - GPU en servidor?

Práctica 5: ¿Qué significa escalar un LLM?

Meta: observar empíricamente el efecto de:

- aumentar el tamaño del modelo (capacidad),
- aumentar el tamaño de datos (señal),
- y el costo computacional (tiempo/memoria).

Práctica 5 LLMs

Hipótesis de trabajo

- Modelos más grandes suelen lograr menor pérdida.
- Con pocos datos, el modelo grande puede sobreajustar.
- El costo crece de forma no lineal.

Pregunta: ¿cuándo deja de valer la pena escalar?

Diseño experimental

Eje A (modelo):

- pequeño, mediano, grande (según CPU/GPU)

Eje B (datos):

- 25 %, 50 %, 100 % del texto

Medir:

- val_loss y val_ppl
- tiempo total

Qué reportar

- Tabla: (tamaño modelo, % datos) $\rightarrow$ val_loss, val_ppl, tiempo
- Gráfica opcional: val_loss vs tamaño de modelo
- Discusión: rendimientos decrecientes

Entregables

- Comparativo de al menos 6 corridas (3 tamaños x 2 tamaños de datos)
- Conclusión técnica: ¿qué escala conviene en CPU vs GPU?
- Recomendación: próxima acción (más datos, más modelo, o mejor entrenamiento)

Discusión

- ① Modelo grande + pocos datos: ¿mejora o empeora? ¿por qué?
- ② ¿Qué parte del costo crece más: capas, embedding, o datos?
- ③ ¿Dónde ves rendimientos decrecientes (mejora pequeña vs costo grande)?
- ④ Si tu objetivo fuera “mejorar val_loss”, ¿invertirías primero en:
 - más datos,
 - más parámetros,
 - o mejor ajuste de entrenamiento?

Práctica 6: Entrenar con recursos limitados

Meta: aplicar técnicas que permiten entrenar modelos más grandes o con batches efectivos mayores, usando el mismo hardware.

Se estudiarán:

- Gradient Accumulation
- Mixed Precision (AMP)
- Gradient Checkpointing

Gradient Accumulation

Idea: simular un batch grande sin cargarlo a memoria.

- Ejecutar k mini-batches
- Acumular gradientes
- Hacer un *optimizer step* cada k iteraciones

Mixed Precision (AMP)

Idea: usar menor precisión numérica para:

- acelerar entrenamiento (en GPU)
- reducir memoria

Se utiliza *autocast* y *grad scaler*.

Gradient Checkpointing

Idea: ahorrar memoria:

- no guardar activaciones de todas las capas
- recomputarlas durante *backprop*

Trade-off: menos memoria, más tiempo.

Métricas a registrar

- `val_loss` y `val_ppl`
- tiempo por época
- (si aplica) memoria máxima GPU
- estabilidad: NaN / divergencia

Entregables

- Tabla comparativa: baseline vs técnicas eficientes
- Conclusión: qué técnica conviene según restricciones
- Recomendación: configuración final para entrenar “más grande”

Proyecto integrador: Diseña tu mini-LLM

Meta: integrar arquitectura + datos + entrenamiento + evaluación.

Cada equipo entregará:

- un modelo entrenado (mini-LLM),
- una demostración de generación,
- un reporte técnico breve.

Requisitos mínimos

- Tarea: *next-token prediction*
- Dataset textual definido y documentado
- Modelo: baseline (GRU) o mini-Transformer causal
- Entrenamiento con cross-entropy
- Evaluación con `val_loss` y `val_ppl`
- Generación con temperatura (al menos 3 valores)

Requisitos de ingeniería

- Reproducibilidad: semillas y configuración guardada
- Registro de resultados: tabla de métricas y tiempos
- Análisis de fallas: al menos un problema observado y su causa probable

Extensiones sugeridas (opcionales)

- Comparar 2 arquitecturas (GRU vs Transformer)
- Probar 2 tamaños de modelo (small vs medium)
- Técnicas eficientes: acumulación, AMP o checkpointing
- Experimento de contexto: L pequeño vs grande

Demostración (demo)

La demo debe incluir:

- 1 prompt fijo (común para el equipo)
- 3 generaciones (temperatura baja/media/alta)
- Comentario: coherencia, errores y repeticiones

Criterios de evaluación

Se evaluará:

- Correctitud del pipeline (datos $\rightarrow$ tokens $\rightarrow$ entrenamiento)
- Calidad del análisis (métricas + interpretación)
- Evidencia experimental (comparaciones y conclusiones)
- Claridad de comunicación (reporte y demo)

Síntesis

En este tema se abordó:

- La arquitectura interna de los LLMs
- El papel central de la atención
- El proceso completo de entrenamiento
- Retos computacionales y de datos

Idea clave

El poder de los LLMs no proviene solo del modelo, sino de la combinación de:

- Arquitectura
- Datos
- Escala

Conexión con el siguiente tema

El siguiente tema abordará:

- Fine-tuning de LLMs
- Adaptación a tareas específicas
- Aplicaciones prácticas